BUILDING AI INFRASTRUCTURE · PART 2 OF 7

Part 2: The Foundation on Your Laptop

Chapter summary for beginners

In one sentence: run all 8 building blocks on your own laptop, for free, with the same tools you'll later use in every cloud.

Video length: about 10 minutes · **Code:** nawex-observability-case-study (folder `demo/`) · **Cost:** $0

Why start on a laptop

The cloud charges you while you learn. On a laptop you can break things, rebuild them and break them again, and it costs nothing. Every tool here is the same one you'll use on AWS, Azure, Google Cloud and on-prem. Only the machines underneath change.

The 8 blocks on a laptop

#	Block	What plays the role locally
1	Compute	Your laptop's CPU (no GPU needed yet)
2	Data and storage	**MinIO** (speaks the same language as Amazon S3) + **Postgres**
3	Networking	kind's built-in network + port-forwards
4	Orchestration	**kind**: a real Kubernetes cluster running inside Docker
5	MLOps	**MLflow**: experiment tracking + model registry
6	Serving	**KServe** InferenceService
7	Observability	Prometheus, Grafana, Loki (logs), OpenTelemetry
8	Guardrails	Simple local passwords (admin/admin). Fine for practice, never for production

What you need

- Docker Desktop with **at least 6 CPUs and 10 GB of memory** (Docker settings → Resources).
- Tools: kind, kubectl, helm, jq, curl, git, docker compose v2. Run `make doctor` in this repo to check.

Build it: one command

```
git clone https://github.com/Here2ServeU/nawex-observability-case-study
cd nawex-observability-case-study/demo
make up      # about 5 minutes the first time; watch each block report "ready"
make demo    # opens Grafana, Alertmanager, Prometheus and MLflow
```

If something fails, the most common cause is not enough memory for Docker. Raise it and run `make up` again.

Tour: one stop per block

1. `kubectl get pods -A`: every block is just a group of pods.
2. **MLflow** at `localhost:5001`: runs, metrics and the model registry (the recipe book).
3. `kubectl get inferenceservice -A`: the model being served, and whether it's ready.
4. **Grafana** at `localhost:3000`: request rate, errors and latency, plus a model dashboard.
5. **Prometheus**: the raw numbers and the alert rules behind the dashboards.

Break it on purpose (the SRE habit)

SLO burn. Run `make scenario-slo-burn`. Errors climb, the **error budget** (how much failure you're allowed this month) drops fast, and the fast-burn alert fires in about 2 minutes. Recover with `make scenario-slo-burn-stop`.

> You should only trust an alert you've actually watched fire.

Data drift (unique to AI). Run `make scenario-drift`. The incoming data stops looking like the training data, and the model quietly gets worse while nothing crashes. The **PSI** (population stability index) score rises, and the drift alert fires in about 3 minutes. Respond by retraining, or roll back with `make scenario-rollback`.

Laptop vs. the real thing

Stays the same everywhere	Changes in the cloud
Kubernetes APIs and kubectl	Real GPUs, in a pool that scales to 0
Helm charts	S3 / Blob / GCS instead of MinIO
MLflow, KServe	Load balancers, private networks
Prometheus, Grafana, OpenTelemetry	Real identity, secrets and budgets

Clean up: `make down` deletes the cluster, and your laptop is back to normal.

Words to know

- **kind:** “Kubernetes in Docker”, a full cluster on your laptop.
- **Pod:** the smallest thing Kubernetes runs, made of one or more containers.
- **Error budget:** the amount of failure an SLO allows (99.5% success means a 0.5% budget).
- **Burn rate:** how fast you’re spending the error budget compared with normal.
- **Data drift:** real-world input gradually differing from the training data.
- **PSI:** a score for how far today’s data has moved from the training data.

Check yourself

1. Which tool plays the role of Amazon S3 on a laptop?
2. Why fire alerts on purpose?
3. Why is data drift dangerous even though nothing crashes?
4. Name two things that change when you move to the cloud.

Answers: 1) MinIO. 2) So you know the alert works before a real outage. 3) The model silently gives worse answers. 4) Any two of: real GPUs that scale to zero, managed storage, load balancers and private networks, real identity, secrets and budgets.

Next: Part 3, the same platform on AWS with EKS and GPUs. Set your AWS budget alert and request GPU quota now.

Emmanuel Naweji · emmanuelnaweji.com · transformed2succeed.com
Code: github.com/Here2ServeU/building-ai-infra · Series guide: docs/series/README.md